

COMICCRAFT

PROJECT PLANNING PHASE

AI Comic Story Creator using Gemini Models

This document presents the Project Planning Phase for ComicCraft, based on the provided project documentation. ComicCraft is a web-based application that generates personalized comic stories, dialogues, illustrations, and downloadable PDF comics from user-provided prompts.

1. Project Overview

ComicCraft combines a FastAPI web application with Google Gemini models and Stable Diffusion to automate the comic-creation workflow. Users provide a story prompt, character name, setting, tone, and art style. The system generates a panel-by-panel outline, narration and dialogue, comic-style illustrations, a preview, and a downloadable PDF.

2. Project Objectives

- Create personalized comic stories from simple user prompts.
- Generate structured five-panel comic outlines automatically.
- Produce detailed narration and character dialogue for each panel.
- Generate comic-style illustrations using Stable Diffusion.
- Provide an interactive web interface for entering preferences and reviewing results.
- Allow users to export the completed comic as a multi-page PDF.
- Provide API endpoints for form-based and JSON-based comic generation.
- Maintain a modular architecture that can support future enhancements.

3. Scope of the Project

In Scope:

- Frontend input form using HTML, CSS, and Jinja2.
- FastAPI backend and route handling.
- Gemini Flash for structured comic panel outlines.
- Gemini Pro for detailed narration and dialogue.
- Stable Diffusion for comic-style image generation.
- Panel layout construction.
- PDF export using FPDF.
- Local development and deployment testing.
- API testing through FastAPI's /docs interface.

Current Limitations / Out of Scope:

- The documented version focuses on single-comic generation.
- User account management is not part of the current version.
- Cloud deployment is identified as a future possibility rather than the current deployment target.

- Future features such as user profiles, comic libraries, and multi-page story arcs are not included in the current scope.

4. Functional Requirements

ID	Requirement	Description
FR-01	User Input	The system shall collect story prompt, main character name, setting, story tone, and art style.
FR-02	Outline Generation	The system shall generate a structured five-panel comic outline.
FR-03	Story Generation	The system shall generate narration and character dialogue from the panel outline.
FR-04	Image Generation	The system shall generate an illustration for each comic panel.
FR-05	Comic Preview	The system shall display generated panels sequentially with title, image, description, captions, and narration.
FR-06	PDF Export	The system shall compile the comic into a downloadable multi-page PDF.
FR-07	JSON API	The system shall support comic generation through a JSON API request.
FR-08	Image Testing	The system shall provide a route for testing image generation independently.
FR-09	Error Handling	The backend shall handle unexpected failures using try-except blocks and FastAPI HTTPException.

5. Non-Functional Requirements

- Usability: The interface should be simple and accessible to non-artists and casual users.
- Maintainability: AI generation, image generation, layout, and export functions should remain modular.
- Responsiveness: The frontend should use a clean layout that works across different screen sizes.
- Reliability: The application should validate inputs and gracefully handle generation or API errors.
- Security: Sensitive Gemini and Hugging Face credentials should be stored using environment variables.
- Testability: API endpoints should be accessible through FastAPI's interactive documentation.

6. Planned Technology Stack

Technology	Planned Use
Programming Language	Python
Web Framework	FastAPI
ASGI Server	Uvicorn
Frontend	HTML, CSS, Jinja2

Text Generation	Google Gemini Flash and Gemini Pro
Image Generation	Stable Diffusion through Hugging Face Diffusers
ML Framework	PyTorch
PDF Generation	FPDF
Image Processing	Pillow
Environment Management	Python virtual environment / pip
Version Control	Git

7. System Architecture Plan

The planned architecture consists of three primary components:

- Frontend Layer: HTML, CSS, and Jinja2 templates collect user input and display the generated comic.
- Backend Layer: FastAPI manages routes, validation, AI workflow coordination, layout construction, and PDF export.
- AI Integration Layer: Gemini models generate structured and creative text while Stable Diffusion generates panel illustrations.

Planned generation flow:

- User enters comic preferences.
- FastAPI receives and validates the request.
- Gemini Flash generates the five-panel outline.
- Gemini Pro expands the outline into narration and dialogue.
- Stable Diffusion generates an image for each panel.
- The layout builder matches images with their corresponding story content.
- The PDF exporter creates the final downloadable comic.
- The frontend displays the completed comic preview and export result.

8. Planned Project Modules

Module	Planning Responsibility
gemini_flash.py	Generate the structured five-panel comic outline.
gemini_pro.py	Generate detailed comic narration and character dialogue.
image_generator.py	Generate and save comic illustrations from image prompts.
layout_builder.py	Organize images and story content into panel layouts.
exporters.py	Compile comic panels into a multi-page PDF.
routes.py	Handle application routes and coordinate the complete generation workflow.
main.py	Provide the FastAPI application entry point.
templates/	Contain index.html, comic_preview.html, and export_success.html.
static/	Store generated panel images and exported PDFs.

9. Development Milestones

Milestone	Area	Planned Activities
Milestone 1	Model Selection and Architecture	Research/select generative AI models, define architecture, and

		set up the development environment.
Milestone 2	Core Functionalities Development	Develop outline generation, story generation, image generation, layout building, PDF export, and FastAPI input processing.
Milestone 3	routes.py Development	Implement the main application routes and integrate the AI workflow.
Milestone 4	Frontend Development	Build responsive HTML/CSS pages and integrate dynamic Jinja2 templates.
Milestone 5	Deployment	Prepare local deployment, configure environment variables, launch the server, and verify the workflow.

10. Development Environment Plan

- Install Python and pip.
- Create and activate a project-specific virtual environment.
- Install FastAPI, Uvicorn, Jinja2, python-multipart, Google Generative AI, Diffusers, Transformers, FPDF, Pillow, and Accelerate.
- Configure Gemini and Hugging Face credentials through environment variables.
- Create the application, templates, static files, and AI utility modules.
- Run the application with Uvicorn using the documented local command.
- Use the browser interface and /docs endpoint to verify functionality.

11. Testing Plan

- Input Validation Test – verify all required comic fields are accepted correctly.
- Outline Generation Test – verify a five-panel structured outline is produced.
- Story Generation Test – verify narration and dialogue are generated for the outline.
- Image Generation Test – verify images are generated and saved for each panel.
- Layout Test – verify each generated image is matched with the correct story panel.
- PDF Export Test – verify the complete comic is compiled into a readable PDF.
- API Test – verify /generate-comic/json returns the expected layout and PDF path.
- Image Utility Test – verify /test-image independently generates an image.
- UI Test – verify the homepage, comic preview, and export success pages render correctly.
- Error Handling Test – verify failures are handled without crashing the application.

12. Security and Configuration Plan

- Do not hard-code API keys or tokens in source files.
- Store GEMINI_API_KEY and HF_API_KEY in environment variables or a .env file.
- Keep credentials outside version control.
- Validate user inputs before passing them into the generation workflow.
- Use controlled file naming when saving generated images and PDF files.

13. Expected Deliverables

- Complete ComicCraft source-code project.
- FastAPI backend with required routes.
- Gemini-based story and outline generation modules.
- Stable Diffusion image generation module.
- Comic layout and PDF export modules.
- Jinja2-based frontend templates.
- Static image and export directories.
- Requirements/dependency configuration.
- Local deployment and testing setup.
- Downloadable AI-generated comic PDF.

14. Project Risks and Mitigation

Risk	Mitigation
AI API failure or unavailable credentials	Use environment variables, clear error handling, and independent testing of AI modules.
Slow image generation	Test image generation separately and monitor generation workflow performance.
Inconsistent story/image outputs	Use structured panel outlines and pass consistent panel-specific prompts through the workflow.
Dependency conflicts	Use a dedicated virtual environment and requirements file.
File/export errors	Validate generated file paths and test PDF creation independently.
Poor user experience	Keep the interface simple, responsive, and organized around a direct prompt-to-preview flow.

15. Success Criteria

- A user can submit a valid comic idea and preferences through the web interface.
- The application produces a structured five-panel outline.
- Narration and dialogue are generated for the panels.
- Illustrations are generated for the panels.
- The generated comic is displayed correctly in the preview page.
- The complete comic can be exported as a PDF.
- The documented API routes work during local testing.
- The project runs successfully in an isolated Python environment.

16. Planning Summary

The Project Planning Phase establishes a clear implementation path for ComicCraft. The project will use FastAPI as the backend framework, HTML/CSS/Jinja2 for the frontend, Gemini Flash and Gemini Pro for structured and creative text generation, and Stable Diffusion for illustrations. Development will proceed through model and architecture selection, core functionality implementation, route development, frontend integration, and local deployment/testing. This plan follows the structure and technical direction of the provided ComicCraft project documentation.

Source basis: ComicCraft - AI Comic Story Creator using Gemini Models (provided project documentation).